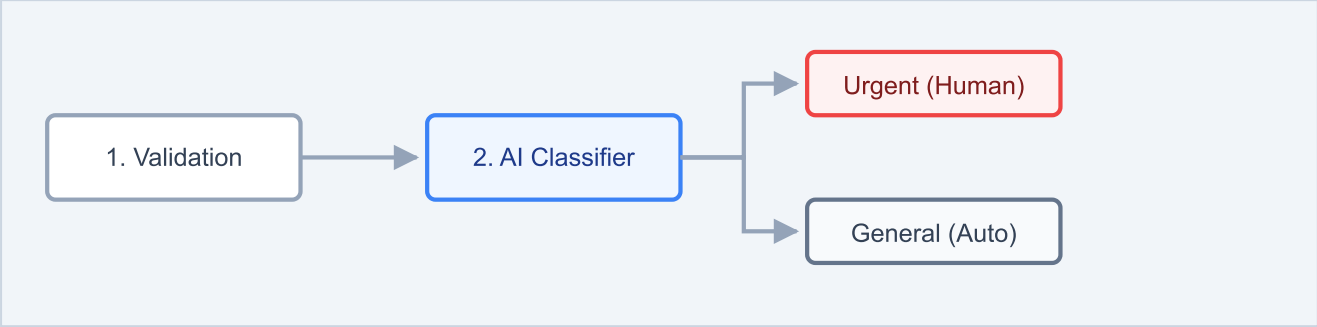
Practical 39: AI Workflow with Conditions

Categorizing Support Requests with Human Oversight

Building an AI-driven support workflow requires combining a natural language classifier with strict conditional routing. This ensures efficient automation while maintaining a safety net for critical or confusing issues. The system operates in three distinct stages to process incoming tickets reliably.

Workflow Architecture & Rules

- **Stage 1 - Validation:** Checks if the incoming payload is readable text. Invalid input returns an error message.
- **Stage 2 - AI Classification:** Analyzes the text and assigns one of three primary labels: Urgent, General, or Unclear.
- **Stage 3 - Conditional Routing:**
  - *Urgent requests:* Flag for immediate human review.
  - *General requests:* Prepare a standard automated response.
  - *Unclear requests:* Ask the user for clarification.



Key Learning Outcomes

- **Conditional Logic:** Traditional conditional blocks serve as an unbending rulebook. While AI handles the nuance of text understanding, standard code handles execution to prevent critical failures.
- **Classification:** The AI acts strictly as a labeler rather than a conversational agent, keeping the system predictable.
- **Human Oversight:** High-stakes edge cases bypass automation entirely, mitigating the risk of AI hallucinations during crises.

Python Implementation

This script demonstrates the rule integration using a mocked AI classification function and standard control flow.

```
def process_support_request(request_text):
    # Stage 1: Validation
    if not request_text or not isinstance(request_text, str) or
len(request_text.strip()) == 0:
        return {"status": "error", "message": "Invalid input provided."}

    # Stage 2: AI Classification
    category = simulate_ai_classification(request_text)

    # Stage 3: Conditional Logic and Routing
    if category == "Urgent":
        # Rule: Flag for immediate human review
        return {
            "status": "escalated",
            "action": "Route to Human Agent",
            "priority": "High",
            "data": request_text
        }

    elif category == "General":
        # Rule: Prepare a standard response
        return {
            "status": "resolved",
            "action": "Send Automated Response",
            "message": "We have received your request and will process it
shortly."
        }

    elif category == "Unclear":
        # Rule: Ask for clarification
        return {
            "status": "pending",
            "action": "Request Clarification",
            "message": "Could you please clarify your issue?"
        }

    else:
        return {"status": "error", "message": "Failed to categorize request."}

def simulate_ai_classification(text):
    text = text.lower()
    if "server down" in text or "urgent" in text:
        return "Urgent"
    elif "password" in text or "billing" in text:
        return "General"
    else:
        return "Unclear"
```